B. Accessibility & Inclusive Design

Standalone capability brief · Suggested module: **Accessibility SDK + Design System Standards**

Purpose

Make accessible behavior a platform default, so product teams can deliver usable experiences for keyboard users, screen-reader users, people with low vision, and people sensitive to motion or other interaction patterns.

1. Scope and capability areas

Keyboard navigation

Provide predictable tab order, visible focus, keyboard-operable controls, skip links, and escape behavior for overlays and dialogs.

Screen-reader semantics

Use correct names, roles, states, relationships, live-region announcements, and accessible descriptions.

Color and visual presentation

Define contrast requirements, non-color status indicators, zoom/reflow behavior, and support for high-contrast and reduced-motion preferences.

Forms and validation

Associate labels and help text with fields, announce errors, summarize validation failures, and move focus to useful recovery points.

Testing and compliance

Provide automated checks plus manual testing guidance. Treat automation as a useful signal, not proof of full conformance.

User preferences

Respect system and product preferences for font scaling, reduced motion, contrast, and other supported accessibility settings.

2. Shared contracts and standards

- Every shared component documents keyboard behavior, accessible name, states, focus behavior, and known limitations.
- Accessibility defects have severity levels and release criteria.
- Design tokens must preserve contrast across themes and tenant branding.
- Accessibility preferences are consumed consistently without overriding user intent unexpectedly.

3. Recommended implementation approach

- Build accessible primitives into the design system rather than adding patches to individual screens.
- Include keyboard-only, screen-reader, zoom/reflow, contrast, and reduced-motion tests in release workflows.
- Use semantic HTML and platform-native controls where possible.
- Provide a documented process for exceptions, remediation ownership, and regression prevention.

4. Failure behavior and operational expectations

- If a complex widget cannot provide a complete accessible interaction, provide a simpler accessible alternative rather than shipping a silent dead end.
- Automated test failures should block critical regressions; false positives must be triaged and documented.
- Tenant branding must not be allowed to produce unreadable contrast or hide focus indicators.

5. Suggested operational metrics

- Critical accessibility defects per release
- Automated test coverage of shared components
- Keyboard and screen-reader task completion in manual audits
- Time to remediate high-severity issues

6. Readiness and acceptance checklist

- ■ Core journeys are operable without a mouse.
- ■ Validation errors are programmatically associated with fields and announced appropriately.
- ■ Theme and branding changes preserve contrast and focus visibility.
- ■ Reduced-motion preferences are respected.
- ■ Automated checks are supplemented by periodic manual evaluation.

Ownership note: assign a named team, service owner, support route, versioning policy, and deprecation process before treating this capability as a platform contract.